

Human Brain :

Module - 1

DATE

Biological

Memory

* 10^{10} (10 Billion Neurons) ✓

* Parallel processing

* Hundreds of operations per second

* The Neuron fires when the input exceeds a threshold value

Cell body
or
Soma

Dendrites

Nucleus

Output

Axon

Synaptic

link : link b/w synapse of one neuron & dendrite of other neuron.

Dendrites = i/p.

Nucleus = processing

Output -

Axon = output

Synapses

(info transfers to other neurons)

(=) Perceptron :

Bias

x_1
 x_2
 x_3

w_1
 w_2
 w_3

$\sum_{i=1}^n x_i w_i$

Then
Summation
function

(Activation
Function)

y
(output)

Input

$$y = \sum_{i=1}^n x_i w_i + B$$

x_i = inputs
 w_i = weights

Bias :

used to better fit a model that are not centered towards (0,0)

* Activation function types :-

	O/P	Nature	Usage	Graph
Step	0 to 1	Discrete	Simple	
Sigmoid	0 to 1	Continuous	Binary o/p	
Tanh	-1 to 1	Continuous	Hidden layer	
ReLU	0 to ∞	Continuous	Hidden layer	
Leaky ReLU	-∞ to ∞	Continuous	Hidden layer	

Step

Sigmoid

Tanh

ReLU

Leaky ReLU

classmate

PAGE

Types of Neural Network:

DATE

① Feed forward NN:

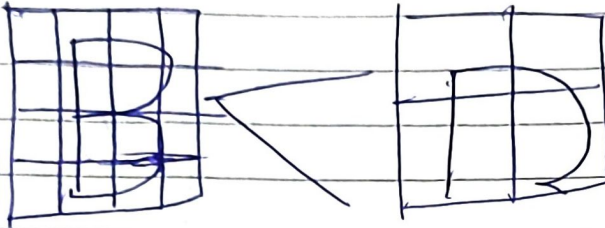
* Only forward signals: i.e. →
only i/p to o/p.

eg: Image classification recognition,
basic classification.

② Convolutional Neural Network:

Looks @ patches / small portions.

eg: Facial recognition, object detection.



③ Recurrent Neural Network: (Has feedback)

Has loops;

Remembers past data.

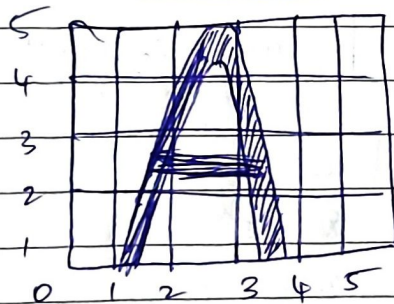
(Has feedback mechanism)

eg: Text prediction, language translation.

⇒ OCR: Optical character recognition

* A technology that converts a
image into machine readable text
using neural network models.

6x6 OCR:



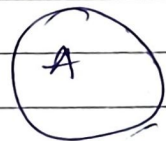
image



Neural Network

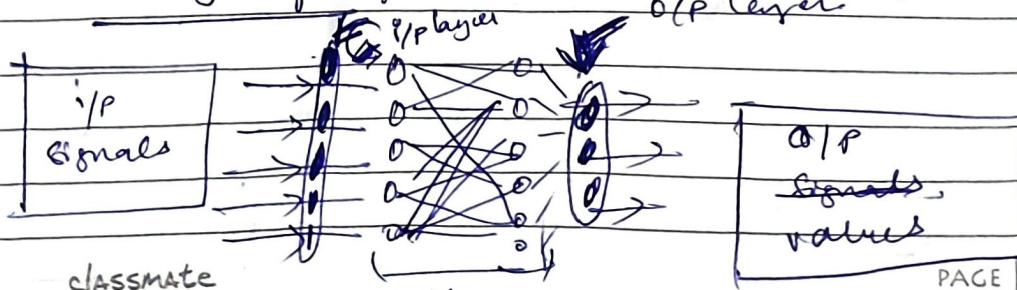


output:



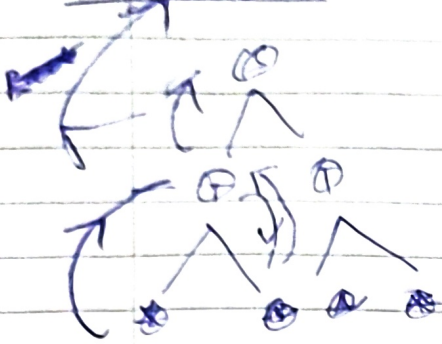
machine
readable text

⇒ Multilayer perceptron:



PAGE

backpropagation :



- Supervised learning technique
- used to minimize the error.

N = neuron

N_w = neuron's i/p weight

N_{out} = Neuron's output

$N_w = N_w + \Delta N_w$

$$\Delta N_{\text{Error factor}} = N_{\text{expected}} - N_{\text{actual}}$$

Learning :

Hebbian Learning

- If two neurons are active at same time then, connection gets stronger.

eg.

Piano learning :

when a man learns piano, he gets improved in playing it as he is using it frequently

$$\Delta W = \eta \cdot x \cdot y$$

$\eta \rightarrow$ small +ve number.

$y \rightarrow$ output

Formulas

formulas :-

Hebbian learning :

Competitive learning :

Boltzman learning :

Sigmoidal Neuron o/p :

Competitive learning

the ~~new~~ neuron learns each time. The Best & fastest neuron is selected to learn.

In o/p layer of Hebbian layer, Neurons compete & learn. The neuron with highest score or activation is declared the winner "winner takes it all!"

Boltzman learning

It uses probabilistic approach to model the data

$$\Delta W = \eta (p^+ - p^-)$$

$$W_{\text{new}} = W_{\text{old}} + \eta (n - W_{\text{old}})$$

$$\Delta W = \eta \cdot x \cdot y$$

$$W_{\text{new}} = W_{\text{old}} + \eta (n - W_{\text{old}})$$

$$\Delta W = \eta (p^+ - p^-)$$

$$f(z) = \frac{1}{1 + e^{-z}}$$

Credit assignment problem :

DATE

- * An agent performs a series of actions.
- * Actions that never helped to reach the goal state are penalized.
- * Credits given to the actions that helped to find the goal.
- * The agent must figure out which actions led to goal & which ~~to~~ never helped at all, with the help of past data.

Learning tasks :

- Supervised - (Classification, regression)
- Unsupervised - (Clustering, Association)

Competitive learning :

g-1) Neuron A

Neuron B

(A)

(B)

$W_A : (0.2, 0.6)$

$W_B : (0.8, 0.4)$

Input
 $x = (0.3, 0.5)$

Measuring distance to choose winner :

$$d_A = \sqrt{(0.3 - 0.2)^2 + (0.5 - 0.6)^2} \quad \begin{matrix} \text{P/P} & \text{(A)} \\ & (0.2, 0.6) \end{matrix}$$

~~$d_A = \sqrt{(0.1)^2 + (-0.1)^2}$~~ $d_A = 0.141$

$$d_B = \sqrt{(0.3 - 0.8)^2 + (0.5 - 0.4)^2} \quad \begin{matrix} \text{P/P} & \text{(B)} \\ & (0.8, 0.4) \end{matrix}$$

$d_A < d_B$
classmate

* Neuron A is selected.

PAGE

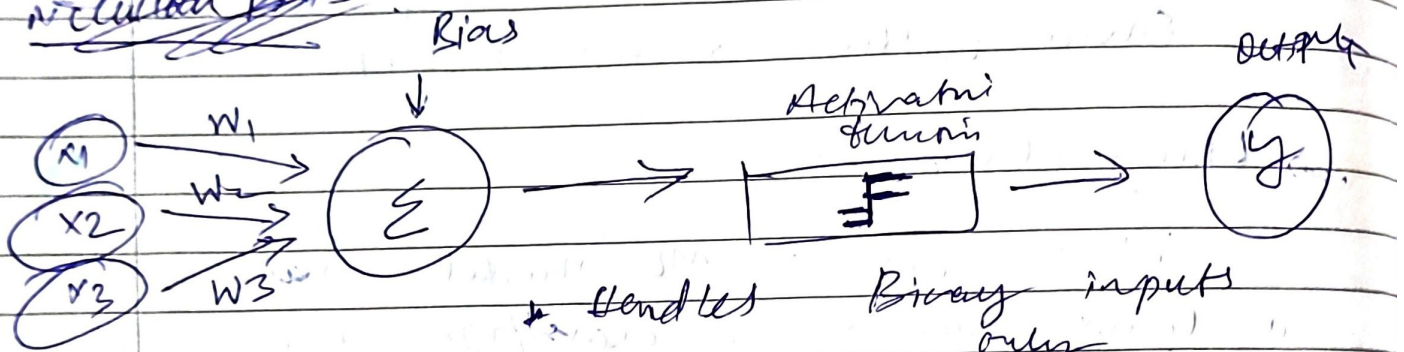
→ Single layer Perceptron :

* simplest form of neural network.

* Primarily used for pattern classification of linearly separable data.

* Perceptron first introduced by :
Rosenblatt - 1953, which
was influenced by McCulloch-Pitts (1943)

McCulloch Pitts :



* Handles Binary inputs only

McCulloch Pitts $\nearrow$ Eg: Yes/No, 0 or 1.

Rosenblatt Model :

~~Can't~~ ~~learn~~ with

Can handle a lot more efficiently
than like real numbers, other numerical
values, when compared to McCulloch-Pitts.

(like real numbers)

$$y = f\left(\sum (w_i \times x_i) + b\right).$$

→ Gradient descent algorithm :

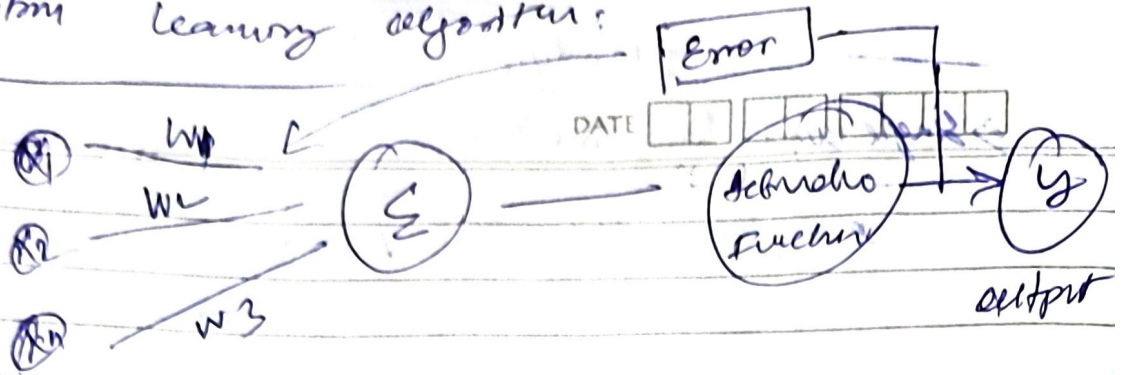
* used to minimize a loss function
by adjusting weights & biases.

(refer upcoming notes below)



next
23/11/23

Perceptron learning algorithm:



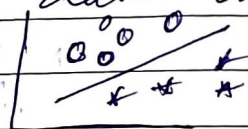
* Initial weights w_1 and $w_2 \Rightarrow 0$ and bias $b = 0$.

$$w_1 = 0 ; w_2 = 0 ; b = 0$$

and set learning rate (η) to a small value such as 0.1.

Linear Classifier:

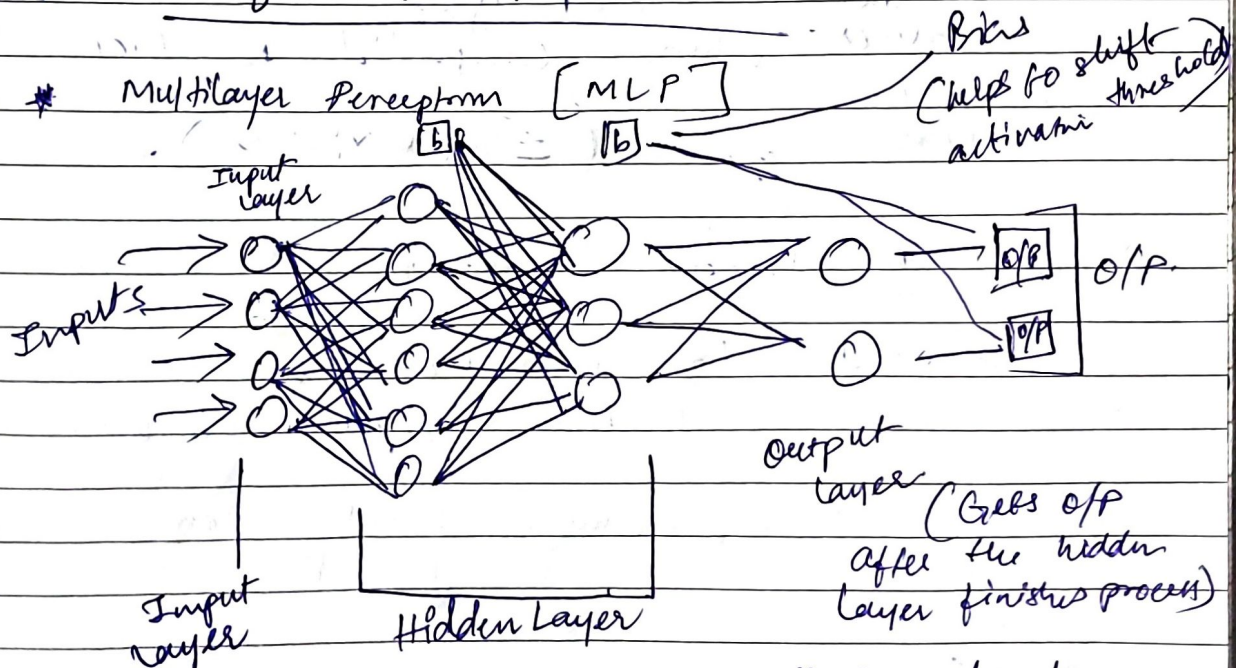
It is a type of ML model that uses a linear function to classify data into two or more classes.



Module - 4

Multilayer Perceptron Networks:

* Multilayer Perceptron [MLP]



(Accepts the raw data as classmate i/p and ~~feeds~~ passes to the hidden layer)

* more amount of hidden layers equals to more computational costs.

Softmax function

used for multiclass
in Neural Networks

classification tasks

DATE

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

→ Neural Network

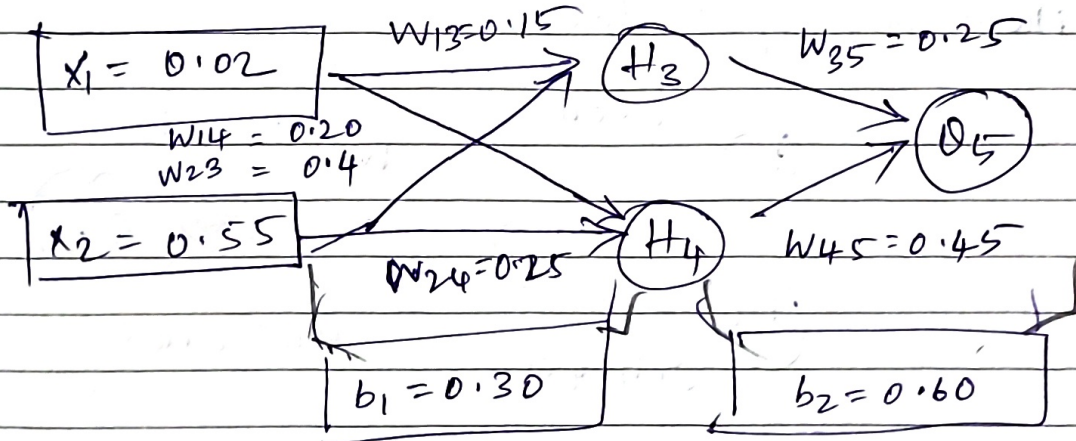
Forward
propagation
phase

Backward
propagation
phase

Info. flows
forward to
compute predictions
and errors

Here, Backpropagation
algorithm computes
the error derivatives
& update network weights

Q For given MLP, perform forward propagation
to calculate the output for tanh activation
function on each layer.



At node H_3

$$\text{net}_{H_3} = (0.02)(0.15) + (0.55)(0.4) + 0.30$$

$$\text{net}_{H_3} = 0.523$$

After applying tanh activation function:

$$\text{out}_{H_3} = \tanh(\text{net}_{H_3}) = \tanh(0.523)$$

$$= \frac{e^{0.523} - e^{-0.523}}{e^{0.523} + e^{-0.523}}$$

$$\begin{aligned} e^{0.523} &= 1.687 \\ e^{-0.523} &= 0.592 \end{aligned}$$

classmate

$$\text{out}_{H_3} = 0.484$$

PAGE

At node H_4 :

$$\text{net}_{H_4} = (0.02)(0.20) + (0.55)(0.25) + 0.30$$

$$\text{net}_{H_4} = 0.4415$$

Applying tanh function, we get:

$$\text{out}_{H_4} = \tanh(-\text{net}_{H_4}) = \tanh(0.4415)$$

$$\text{out}_{H_4} = \frac{e^{0.4415} - e^{-0.4415}}{e^{0.4415} + e^{-0.4415}}$$

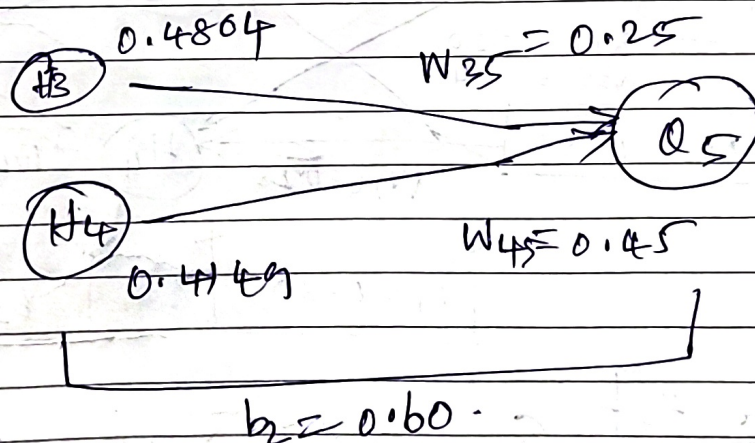
$$\begin{aligned} e^{0.4415} &= 1.555 \\ e^{-0.4415} &= 0.643 \end{aligned}$$

$$\text{out}_{H_4} = \frac{1.555 - 0.643}{1.555 + 0.643}$$

$$\text{out}_{H_4} = \frac{0.912}{2.198} = 0.4149$$

$$\text{out}_{H_4} = 0.4149$$

Now, at output layer:



At Q_5 :

$$\text{net}_{Q_5} = (0.4804)(0.25) + (0.4149)(0.45) + 0.60$$

$$\text{net}_{Q_5} = 0.906$$

classmate

After applying tanh & activation function:

$$\text{out}_{o5} = \tanh(\text{net}_{o5})$$

$$= \tanh(0.906)$$

$$\text{out}_{o5} = \frac{e^{0.906} - e^{-0.906}}{e^{0.906} + e^{-0.906}} = \frac{2.744 - 0.404}{2.744 + 0.404}$$

$$\begin{aligned} e^{0.906} &= 2.744 \\ e^{-0.906} &= 0.404 \end{aligned}$$

$$= \frac{2.34}{3.148} = 0.743$$

Output = 0.743

→ Delta-Learning Rule:

- * Adjusts network weights to minimize error b/w desired & actual o/p.
- * Used to update weights of o/p neurons.
- * Based on gradient descent minimization of error.

Back propagation algorithm:

• Used to train MLPs.

Steps:

- Forward pass
- Error calculation
- Backward pass
- Weight update

this is how backpropagation works.

$$\Delta W_{ij} = -\eta \frac{\partial E}{\partial W_{ij}}$$

special slides

Network Learning Rules in Neural Network:

- * Hebbian - It identifies data to modify the weights of nodes of a network.
- * Perceptron - Network starts its learning by assigning a random value to each weight.
- * Delta learning - modification is synaptic rule weight of a node is product error & i/p.

⇒ Hebbian Learning Rule: (Designed for unsupervised)

- * First learning rule in NN.
- * By Donald Hebb in 1949.
- * Basic principle: "Neurons that fire together wire together". (Neurons that are active at the same time, has strong connection)
- * Both i/p are positive → strong +ve weight
- * Both i/p are negative → strong -ve weight

$$w_{ij} = x_i \cdot x_j \rightarrow \text{learning rate}$$

⇒ Perceptron Learning Rule:

$$y = f\left(\sum_{i=1}^n x_i w_i + B\right)$$

$$x_i = \text{i/p}$$

$$w_i = \text{weights}$$

$$B = \text{Bias}$$

$$f = \text{Activation function}$$

$$y = \text{output}$$

Delta Learning rule

- refer previous page.

⇒ Boltzmann machine:

* Energy based model.

* learns by minimizing energy function

Applications:

* Image recognition

* NLP.

=> Perceptron problem :

i/p	weight
$x_1 = 0$ or 1	$w_1 = 0.7$
$x_2 = 0$ or 1	$w_2 = 0.6$
$x_3 = 0$ or 1	$w_3 = 0.5$
$x_4 = 0$ or 1	$w_4 = 0.3$
$x_5 = 0$ or 1	$w_5 = 0.4$

Threshold = 1.5

DATE

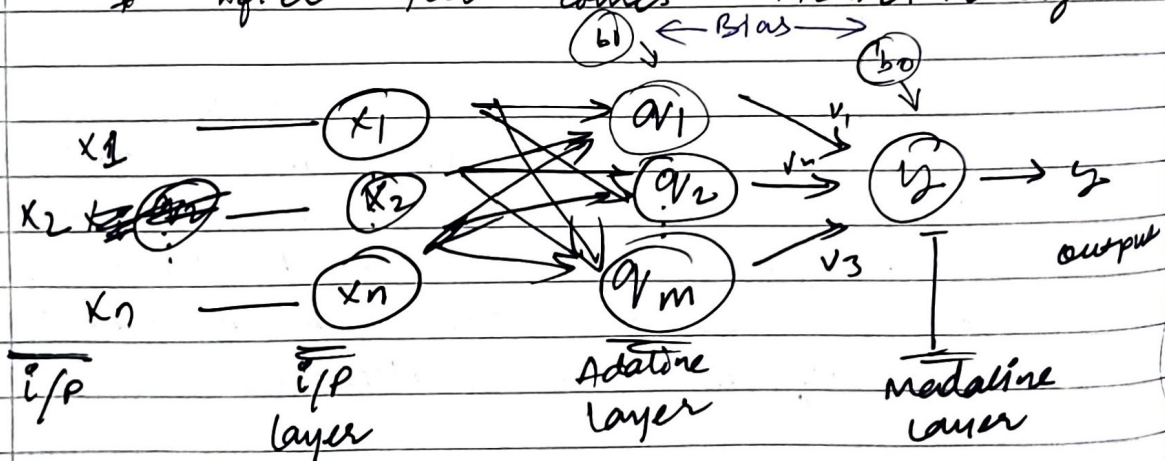
check if num first 5
dines or not

=> Adaptive linear combiner :- (ADALINE)

- * An adaptive linear neuron or "ADALINE".
- * single-layer Neural Network.
- * Combines multiple input signals to produce an output that closely matches the desired o/p signal.

=> Multiple Adaptive Linear Neuron (MADALINE)

- * As the name suggests, it is combination of multiple ADALINEs = MADALINE.
- * ADALINE layer present b/w the i/p & ~~output~~ acts as hidden layer.
- * After that comes MADALINE layer.



Limitations of perceptron.

- * Requires labelled dataset.
- * Limited to linearly separable problems.
- * Complex problems requires more hidden layers.

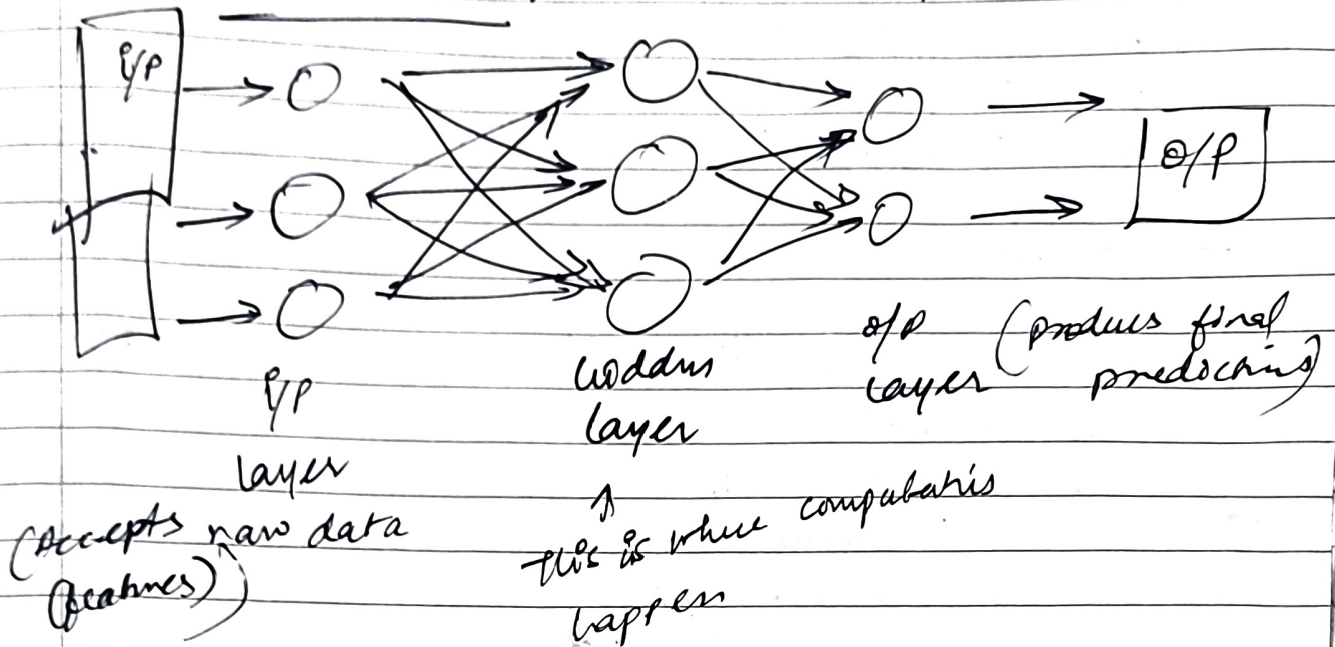
According to special slide:

UNIT - IV:

DATE

--	--	--	--	--	--	--	--

⇒ MultiLayer Perceptrons: consists of hidden layers.



⇒ Gradient Descent:

- * Iteratively adjusts model's parameters to reduce the difference b/w predicted & Actual values, improving the model's performance

⇒ Generalized Delta Learning rule (GDLR)

- * Foundation of how error backpropagation works in training Multi-layer Neural Networks.
- * It generalizes Delta rule to handle MLP, differentiable Neural Networks.

Steps:

Forward Pass → Calculate Error → Backward Pass

↓
Update weights

— UNIT - 5 —

→ Associative Memory:

- * A special type of memory.
- * It stores information, so it can be recalled later, using just a part of its content; not address/locus.
- * In simpler terms, searching for a saved contact in a phone just by using their first name.

* 5 - types:

1. Auto associative memory:
Remembers a pattern & recalls the same pattern.
2. Hetero - Associative Memory:
Links one pattern to a very different pattern.
Eg: $ip \rightarrow dog$; $ofr \rightarrow Perro$ (Dog in Spanish)
3. Linear Associative Memory:
Uses linear operations to map input & opp.
Eg: If $ip \rightarrow 2$ then we use the rule: $3 \times ip$ (for example)
 $opp \rightarrow 6$.
4. Distributed Associative Memory:
Spreads & distributes data across many connections, so, even a part is lost, recall still works.
Eg: Remembering a whole face even if only half is visible.
5. Adaptive Associative Memory:
Learns & adapts its connections with new data, improving over time.

→ Numericals: ————— ANN —————

Design a Bipolar Auto associative (Hopfield) memory to store $P_1 = [1, 1, 1, 1]^T$ and $P_2 = [-1, 1, 1, -1]^T$. Evaluate responses to 1 bit noise...

Step 1 → Outer product: Hebbian learning rule:
Set diagonals = 0 for W_{new} (weight)

Step 2 → Find $P_1 P_1^T$ and $P_2 P_2^T$

$$P_1 P_1^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix}$$

1x4

$$= \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

4x4

$$P_2 P_2^T = \begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \end{bmatrix} \begin{bmatrix} -1 & 1 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

classmate

Step-3

$$W_{\text{row}} = P_1 \cdot P_1^T + P_2 \cdot P_2^T$$

DATE

$$W_{\text{row}} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

$$W_{\text{row}} = \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}$$

Outer product
Hebbian rule:
Set diagonals = 0.

$$W_{\text{row}} = \begin{bmatrix} 0 & 0 & 0 & 2 \\ 0 & 0 & 2 & 0 \\ 0 & 2 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{bmatrix}$$

Step-4: Check fixed points. (check whether they are stable or not).

For $P_1 = [1, 1, 1, 1]^T$: $\text{sgn} =$

$P_2 = [-1, 1, 1, -1]^T$: $\text{sgn} =$

$\text{sgn} \rightarrow$ function

$$\text{sgn}(x) = \begin{cases} +1 & \text{if } x > 0 \\ -1 & \text{if } x < 0 \end{cases}$$

$$\text{sgn}(P_1) = \text{sgn}([1, 1, 1, 1]^T) = P_1 \checkmark$$

$$\text{sgn}(P_2) = \text{sgn}([-1, 1, 1, -1]^T) = P_2 \checkmark$$

any one stable

Final answer:

The weighted matrix

$$W = \begin{bmatrix} 0 & 0 & 0 & 2 \\ 0 & 0 & 2 & 0 \\ 0 & 2 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{bmatrix}$$

All formulas:

① Hebbian Learning Rule: (Pioneer Learning)
Fire together, then wires together

$$\Delta W_{ij} = \eta x_i y_j$$

$$\Delta W = \eta x y$$

ΔW = change in weight ($W_2 - W_1$)

η = Learning rate

x = input

y = output

② Delta Learning Rule:

* updates & modifies weights to minimize error b/w desired and actual output,

$$\Delta W = \eta (y_{\text{pred}} - y_{\text{actual}}) x$$

Learning rate.

③ Backpropagation:

① Forward pass

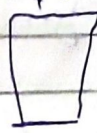
② Find error = (target - actual)

③ backward pass

④ Adjust weights

④ Radford Machine:

* uses probabilistic approach to model the data distribution.
Learns by minimizing cross-entropy function.



Weight Update Rule:

$$\Delta W = -\eta \frac{\partial E}{\partial W}$$

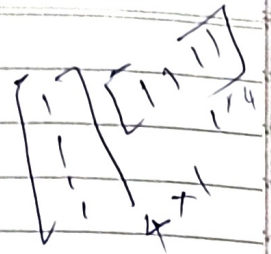
learning rate

Error

DATE

weight

DATE



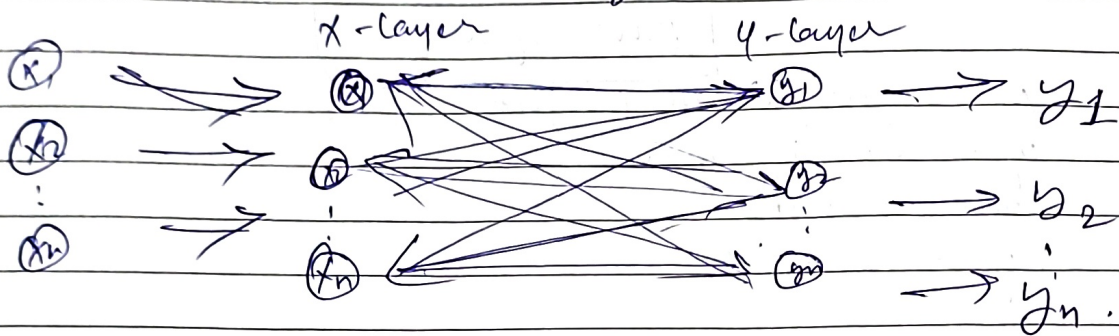
Bidirectional Associative Memory:

Recurrent Neural network.

Allows information flow in both directions

Only 2 layers i/p and o/p

weights are symmetric. ($w_{ji} = w_{ij}$)



Problem to design Hopfield Network:-

$$w_{mn} = \sum_{i=1}^n x_i x_i^T$$

$$w_{mn} = \sum_{i=1}^n (x_{i1} x_{i1}^T + x_{i2} x_{i2}^T + \dots + x_{in} x_{in}^T)$$

Set diagonal = 0

Check stability $\text{sgn}(x_1 x_1^T)$ and $\text{sgn}(x_2 x_2^T)$

$$\text{sgn}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

exactly recovered,

weight matrix $(W) (w_{mn})$

PAGE

Activation Functions

Sigmoidal:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

DATE

--	--	--	--	--	--	--	--	--	--

Tanh:

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

ReLU:

$$f(x) = \max(0, x)$$

Leaky ReLU:

$$f(x) = \max(0, x, x)$$

Softmax:

$$f(x) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Sign:

$$f(x) = \begin{cases} +1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Learning:

Hebbian:

$$\Delta W = \eta x y$$

Delta:

$$\Delta W = \eta (y_{\text{pred}} - y_{\text{actual}}) x$$

Backprop:

$$\Delta W = -\eta \frac{\partial E}{\partial W}$$

Competitive Learning:

$$W_{\text{new}} = W_{\text{old}} + \eta (n - W_{\text{old}})$$